第一篇论文发布执行计划

Oct 12, 2026 · @shangfeng

总览

目标：用 4–5 周，把 v5.8 改成一篇英文测量论文，先发到 Zenodo 拿 DOI，再争取 arXiv；同时向 FreeToken 上游提交一个有数据支撑的 issue。

论文的定位从“提出机制”改为“刻画与界定”：成本模型、它在实测上的验证、链路硬上界，以及为什么调度类机制在这台机器上空间很小。M1–M6 没有收益，作为负结果写进一节，不再是失败。

三条原则

1. **数据冻结。** 以 FTD-20261009-1 加附录 A.5 的轮次为准，整个过程不开新实验。唯一例外：审核发现某个核心数字无法从原始工件复算，先记录，再决定是否补测。
2. **一次只做一个阶段。** 每个阶段有验收标准，过了再进下一个。
3. **Claude Code 负责核对和起草，你负责判断。** 它给出的每一处修改，你看过再接受。

阶段	内容	预计用时	产出
0	冻结数据、准备工作目录	1 天	论文仓库、CLAUDE.md
1	Claude Code 审核数据	3–5 天	数字台账、核对报告
2	改写为英文论文	2–3 周	可编译的 LaTeX 稿
3	向上游提交 issue	1 天，可与阶段 2 并行	GitHub issue
4	发布	Zenodo 1 天；arXiv 视背书而定	DOI、arXiv 编号

阶段 0：冻结数据，准备工作目录（1 天）

这一步结束时，你应该有一个独立的论文仓库，Claude Code 在里面只读原始数据、只写论文文件。

- [ ] 确认服务器上 `/data2/_shared/runs/` 下的原始工件都是只读的；把附录 A.5 的轮次清单封存为 FTD-20261010-1（按你自己的版本规则）。
- [ ] 新建论文仓库，例如 `~/paper-en/`，执行 `git init`。
- [ ] 建好目录：`sections/`（各节 LaTeX）、`figures/`（生成的图）、`data/`（只放从工件导出的 CSV）、`scripts/`（复制 `theory_v55.py`、`theory_v58.py`、`pass_queue_model.py`）、`audit/`（审核结果）、`source/`（放 v5.8 的 HTML 原稿）。

- [ ] 把下面的内容保存为仓库根目录的 `CLAUDE.md`。Claude Code 每次启动都会读它。
- [ ] 在仓库目录里启动 Claude Code，先输入：“读一下 CLAUDE.md，用中文复述你理解的规则，不要做任何修改。”确认它理解对了再开始。

```
# 项目：主存卸载 MoE 测量论文（英文版）

## 背景
本仓库把中文预印本草稿 source/FreeToken_MultiGPU_CoExec_v5.8.html
改写为一篇英文测量论文。数据已冻结，不做新实验。

## 硬性规则
1. 原始工件目录 /data2/_shared/runs/ 只读：不修改、不移动、不删除任何文件。
2. 不启动推理服务，不占用 GPU，不运行任何会发请求的脚本。
3. 论文中的每个数字，必须能在 audit/claims.csv 中找到来源（工件路径 + 字段，或推导脚本）。
4. 找不到来源时标记 UNTRACEABLE，不要猜，不要用相近的数代替。
5. 修改论文文件前，先说明要改什么、为什么，等我确认。
6. 论文正文用英文；给我的报告和说明用中文。
7. 论文正文不出现内部编号（batch、R0xx、SHA、FTD、“独审”“用户裁定”等），这些只留在 audit/ 和补充材料里。

## 论文定位
这是一篇“刻画与界定”的论文，不是“提出机制”的论文：
成本模型 → 实测验证 → 链路硬上界 → 调度空间为什么小 → hybrid 越过上界的收益与非无损代价。
多卡部分只作讨论，标明为分析，不报告多卡性能。
```

阶段1：用 Claude Code 审核数据（3–5 天）

目标是让论文里每个数字都能追到原始工件。按顺序把下面五段提示词逐条粘贴给 Claude Code，每段做完、你看过结果，再发下一段。

1.1 建数字台账

```
读取 source/FreeToken_MultiGPU_CoExec_v5.8.html，把正文、表格、图注中出现的每一个数值结论抽取到 audit/claims.csv。
列：claim_id，章节，原句（中文），数值，单位，文中引用的证据名或轮次，是否核心结论(Y/N)。
核心结论指：表 5b、表 12f、两路链路上界 23.9、hybrid 同窗口 A/B、合成文本与真实内容的失误率差、首 token 一遍/两遍两档、锚点切段 2→1。
只抽取，不判断对错。完成后用中文告诉我总条数和核心条数。
```

1.2 逐条追溯到原始工件

```
对 audit/claims.csv 中的每一条，到 /data2/_shared/runs/ 下找到对应的原始文件（只读），从原始数据重新计算该数值。
在 claims.csv 增加三列：artifact_path, recomputed_value, status。
status 取值：MATCH（一致，允许末位舍入差）/ MISMATCH / UNTRACEABLE / DERIVED（由公式推出，留到 1.3 核对）。
先做核心结论，再做其余。不要修改任何原始文件。
把过程和问题写进 audit/trace_report.md（中文）。
```

1.3 复算推导量

```
运行 scripts/ 下的 theory_v55.py、theory_v58.py 和 pass_queue_model.py（纯 CPU 计算，不涉及服务），核对所有 status=DERIVED 的条目，包括：L*（233 与 133）、R_link 两路上界 23.9、表 5a/5b/5c 的理论值、图 4 的排队仿真。
同时检查每个公式的量纲是否一致。
结果更新到 claims.csv，并在 audit/trace_report.md 增加“推导核对”一节。
```

1.4 一致性检查

```
检查同一个量在全文不同位置是否一致（例如失误率、233/133、87K 的各个读数、Strata 版本号 v0.1.39 与 v0.1.41、各表之间的重复数字）。
列出所有不一致之处，写进 audit/consistency.md，每条给出位置、两处的写法、你判断哪个正确及依据。
```

1.5 审核总结

```
汇总 1.1–1.4，写 audit/SUMMARY.md（中文）：
1. 核心结论逐条的状态；
2. 所有 MISMATCH 和 UNTRACEABLE 条目，以及建议：修正 / 删除 / 降级为“观察”；
3. 你认为审稿人最可能质疑的三个地方。
不要修改论文，等我决定。
```

验收标准

- 核心结论 100% 为 MATCH 或推导已核实。
- 非核心数字中，UNTRACEABLE 的一律从论文删除，不补救。
- 如果某个核心结论对不上：先修正数字；修正后结论不成立的，就从论文中拿掉这个结论，而不是补实验去救。

建议：1.2 完成后，让 Claude Code 另开一个子代理，只拿核心结论清单和工件路径独立复算一遍，不看第一次的结果。两次一致，才算真正核实。

阶段 2：改写为英文论文（2–3 周）

阶段 1 的 SUMMARY.md 你确认过之后再开始。目标篇幅 12–16 页，正文只讲结论和证据，内部过程全部移出。

暂定题目：*Expert-Stream Passes and the PCIe Bound: A Cost Model and Measurement Study of Host-Memory-Offloaded MoE Serving on a Consumer GPU*

论文结构与对应的现有材料

节	内容	来自 v5.8
1 Introduction	问题；三个发现；四条贡献	§1 重写
2 Background & Related Work	FreeToken 机制、平台、相关工作	§2、表 3
3 Cost Model	遍数、L*、链路上界、按执行结构的可达值、失误率随内容变化	§4.1–4.4、§4.7、表 5a/5c
4 Methodology	平台、负载（合成文本、真实会话、X/Y）、指标、冷/热配对协议	§3.1、§6 精简
5.1 Model Validation	模型对实测 ±5%	表 5b + 表 12f 合并
5.2 Time to First Token	一遍/两遍两档、实现地板、锚点切段	§7.3–7.4、图 7、图 8、表 9
5.3 Synthetic vs. Real Content	失误率 25% 对 35–39%，合成基准高估解码	§4.7、§12 第 10 条
5.4 Two Streams and the Link Bound	闭环 X/Y，23.9 硬上界	§8.3
5.5 Crossing the Bound: Hybrid	同窗口 +21% / +55%，W4A8 与缓存历史依赖	§7.7
5.6 Why Scheduling Has Little Headroom	M3 上限、M4 上界、替换策略回放、Belady	§5.2、§8.1、§8.3 D15 段
5.7 Gap to Strata	简要分解，标明推算	§8.4 精简
6 Discussion: Multiple GPUs	命题 1–2，明确为分析	§4.6、§5.5
7 Limitations	集中写一次	§12 精简
8 Conclusion		§11 重写
Appendix	正确性（T17）、复现材料	§9–10

删除或移到补充材料：所有轮次编号、SHA、版本变更记录、D10 原型的过程细节、Strata S2–S4 细节、M5 段计数表、多卡组级执行器 G1–G5 的设计细节。

2.1 建 LaTeX 骨架

```
按 CLAUDE.md 的论文定位，在仓库中建立 arXiv 可编译的 LaTeX 骨架：main.tex（article 类，natbib，单栏）、sections/ 下每节一个文件、refs.bib（从 v5.8 参考文献转录，补全 arXiv 编号）。
节结构按我给的表格。先只写各节标题和每节要表达的一句话要点（英文），不写正文。完成后用 pdflatex + bibtex 编译一次，告诉我是否通过。
```

2.2 逐节改写（每次一节）

```
现在写 sections/<节名>.tex。依据：source/ 中 v5.8 对应章节，以及 audit/claims.csv 中已核实的数字。
要求：
1. 用清楚、平实的学术英文，不逐句翻译，按英文论文的逻辑重组；
2. 只用 status 为 MATCH 或已核实 DERIVED 的数字，并在注释中标出 claim_id；
3. 不出现任何内部编号；
4. 每个结论最多一处限定语，所有局限集中到 Limitations；
5. 写完后用中文告诉我：这一节的要点、用了哪些 claim_id、你不确定的地方。
```

2.3 重画图表

```
用 matplotlib 从 data/ 下的 CSV 重新生成论文需要的图（参考 v5.8 的图 2、3、4、7、8 和表 5b/12f 的数据），输出 PDF 到 figures/。
每张图对应一个 scripts/fig_*.py，数据来自 CSV，不手写数值。字号、配色统一，适合黑白打印。
```

2.4 挑刺审稿

```
假设你是 cs.DC 方向一位挑剔的审稿人，没有看过任何背景材料。完整阅读编译后的论文，按“必须修改 / 建议修改 / 小问题”列出意见，写进 audit/review_round1.md（中文）。重点看：贡献是否清楚、结论是否超出证据、和 FreeToken 原文的区别是否讲清、负结果是否有说服力。
```

2.5 投稿前检查

```
检查 arXiv 投稿要求：pdflatex 能在干净的 TeX Live 下编译；包含 .bbl；无绝对路径；图都是 PDF/PNG；无未定义引用；摘要 ≤ 1920 字符。列出问题并修复（修复前先告诉我）。
```

验收标准：编译无错误；所有图由脚本生成；2.4 的“必须修改”全部处理；你自己从头读一遍，能用两三句话说出论文的结论。

阶段 3：向 FreeToken 上游提交 issue（1 天，可与阶段 2 并行）

先提 issue，不要直接提 PR。改默认阈值涉及取舍（hybrid 更快，但 CPU 路径是 W4A8），应该先让维护者表态。这一步不需要写代码，只需要把数据讲清楚。

操作步骤

1. 用 GitHub 账号登录，打开上游仓库 `FlashML-org/FreeToken`（以论文参考文献 [9] 为准）。
2. 在 Issues 里先搜索 `benchbw`、`hybrid threshold`、`auto strategy`，确认没有人报过同样的问题。有的话，在那个 issue 下补充你的数据即可。
3. 点 **New issue**，标题和正文用下面的草稿。发之前把方括号里的内容替换掉，并删除任何公司名、内部路径和服务器地址。
4. 发完之后不用催。一周内没有回复也正常。
5. 另外两件低成本的事：在 PR #578 和 #577 下各留一条评论，报告你在 RTX 4090 上的验证结果（下面附了 #578 的评论草稿）。这对维护者有用，也让他们知道你是谁。

Issue 草稿（让 Claude Code 帮你核对每个数字和 `benchbw.py` 中阈值的确切位置后再发）

```
Title: Auto strategy picks offload on a host where hybrid decodes 21–55%
faster (CPU/PCIe ratio 1.96 vs. 2.0 threshold)

## Summary
On my machine, the automatic strategy selection in `benchbw.py` chooses
`offload`,
because the measured CPU/PCIe bandwidth ratio is 1.962, just under the
"CPU bandwidth > 2 × PCIe bandwidth" threshold. In a same-window A/B test,
`--moe-strategy hybrid` decoded 21% faster single-stream and 55% faster per
stream with two concurrent requests.

## Environment
- GPU: 1 × RTX 4090 24 GB, PCIe 4.0 x16
- CPU: 2 × Xeon Silver 4310 (AVX-512 VNNI, no AMX), dual-socket NUMA
- Model: Qwen3.8-Flash-Next NVFP4
- FreeToken: fork of upstream [d3512b4] with PRs #577 and #578 cherry-picked
- Measured expert H2D bandwidth: 24.3 GB/s

## Measurement (same window, same build)
Services started in order offload → hybrid → hybrid (fresh) → offload.
Prompts of ~1.2K input tokens, 256 output tokens, temperature 0.
The only change for hybrid was `--moe-strategy hybrid` plus CPU pinning.

| | offload | hybrid |
|---|---|---|
| Single stream (tok/s) | 29.62–30.19 | 35.55–36.59 |
| Two streams, per stream (tok/s) | 16.81–16.98 | 25.96–26.27 |

The two offload services differed by 0.20% in two-stream throughput, so
the gap is not drift.

## Notes
- With two distinct streams, offload is close to its PCIe bound
```

```
  (~23.9 tok/s per stream at the measured miss rate); hybrid exceeds it
  because CPU-computed experts do not cross the link.
- Hybrid is not numerically identical to offload: the CPU path quantizes
  activations to int8 (W4A8), and outputs also depend on cache history
  (a fresh and a warmed service diverge on the same cold request). So this
  is a speed/numerics trade-off, not a free win.

## Question / suggestion
Would you consider one of these?
1. Making the threshold configurable, or documenting it;
2. Deciding by a short end-to-end decode probe instead of a bandwidth ratio;
3. Logging the measured ratio and the chosen strategy at startup, so users
   near the boundary know to try the other one.

Happy to share logs or run additional measurements.
```

PR #578 评论草稿

```
Validated this PR on an RTX 4090 (PCIe 4.0) with Qwen3.8-Flash-Next NVFP4,
cherry-picked onto [d3512b4].

For a cold 32K-token prefill, MoE compute time dropped from 9.42 s to 2.22 s
(4.24×). Output text, chunk boundaries and per-pass H2D bytes were
bit-identical to the unpatched build. Thanks for the work!
```

可选：后续提 PR（只在维护者回复表示欢迎之后）

```
上游 FlashML-org/FreeToken 的维护者在 issue [链接] 中同意 [他们同意的方案]。
请用 gh CLI 帮我：fork 仓库，从上游最新 main 建分支，只做这一处修改，保持风格与现有代码
一致，补一个最小测试或说明，然后起草 PR 描述（英文，引用该 issue）。
每一步执行前先告诉我要运行的命令，等我确认。不要 push，直到我说可以。
```

阶段 4：发布

顺序是先 Zenodo、后 arXiv。Zenodo 不需要任何人批准，论文一公开，成果和时间戳就定下来了；arXiv 拿到背书后再补上，两者不冲突。

4.0 发布前确认

- [ ] 署名单位：沿用“独立研究者”，或写公司名。写公司名需公司书面同意。
- [ ] 如果实验用了公司的服务器或内部负载，确认公司同意公开这些数据；论文和仓库里不出现公司内部信息。
- [ ] 注册 ORCID（免费），Zenodo 和 arXiv 都可以关联它。

4.1 公开仓库与 Zenodo DOI（1 天）

1. 在 GitHub 新建公开仓库，放入：论文 PDF、`scripts/`、`data/` 中脱敏后的 CSV、`README.md`（说明如何从 CSV 复现每张图和每个理论值）。
2. 用 GitHub 账号登录 Zenodo，在 Zenodo 的 GitHub 设置中开启这个仓库。
3. 在 GitHub 上发布一个 Release（例如 `v1.0`），Zenodo 会自动为它生成 DOI。
4. 也可以在 Zenodo 单独上传论文 PDF：资源类型选 Publication → Preprint，许可选 CC BY 4.0，关联 ORCID。
5. 把 DOI 写进论文首页脚注和仓库 README。

4.2 arXiv 投稿与背书

- 主分类 **cs.DC**，交叉列出 **cs.PF** 和 **cs.LG**。CS 是同一个背书域，一次背书即可。
- 注册 arXiv 账号，关联 ORCID，开始一次新投稿并选择 cs.DC，系统会发来带背书码的邮件。
- 找背书人：从论文引用的 arXiv 论文摘要页底部点“Which authors of this paper are endorsers?”。优先顺序：在 issue 里回复过你的 FreeToken 作者 → KTransformers 作者 → Layered Prefill、MoE-Lightning 等相关工作作者。
- **一次只联系一位**，等一周没有回音再联系下一位。不要群发，也不要反复催同一个人。
- 拿到背书后上传 LaTeX 源码（含 .bbl），在 Comments 栏写明 Zenodo DOI 和代码仓库链接。

背书请求邮件模板

```
Subject: arXiv endorsement request (cs.DC): measurement study building on
FreeToken

Dear Dr. [Name],

I am an independent researcher working on LLM inference infrastructure.
I have written a measurement paper that builds directly on [your work, e.g.
FreeToken], and I would be grateful if you would consider endorsing me for
arXiv cs.DC. This is my first arXiv submission.

The paper studies host-memory-offloaded MoE serving on a single RTX 4090.
It derives a cost model based on expert-stream passes and PCIe time,
validates
it within ±5% on real agent workloads, and shows why scheduling mechanisms
have
little headroom under the link bound, while CPU-side expert execution can
exceed it at a numerical cost.

- Paper (PDF): [Zenodo DOI link]
- Code and data: [GitHub link]
- Endorsement code: [six-character code]
[- Related upstream issue: link, if you have one]
```

```
I understand endorsement is not peer review, and I fully understand if you
prefer not to. Thank you for your time either way.

Best regards,
[Your name]
[ORCID link]
```

发之前，让 Claude Code 帮你把第二段改成最贴近对方研究方向的一句话：给 FreeToken 作者强调你复现和扩展了他们的系统，给 KTransformers 作者强调 CPU 路径的结果。

每周检查与卡住时怎么办

每周检查表

- [ ] 第 1 周：阶段 0 完成；阶段 1 的 1.1–1.3 完成
- [ ] 第 1 周：上游 issue 和 #578 评论已发出
- [ ] 第 2 周：阶段 1 的 SUMMARY.md 已确认；LaTeX 骨架能编译
- [ ] 第 2–3 周：第 3、5 节初稿完成（论文的核心）
- [ ] 第 3–4 周：其余各节、图表完成；挑刺审稿一轮并处理
- [ ] 第 4–5 周：投稿前检查通过；Zenodo DOI 拿到；联系第一位背书人

常见风险

情况	信号	怎么做
想补实验	“再测一组就更完整了”	写进论文的 Future Work，不跑
核心数字对不上	1.2 出现 MISMATCH	修正数字；结论不成立就删掉，不补救
英文表达没把握	读起来像翻译	让 Claude Code 按 2.2 重组段落，你只看意思对不对
背书没有回音	一周无回复	换下一位；Zenodo 版本已经公开，不受影响
上游没有回复	issue 无人理	正常现象，不影响论文发布
进度落后	某阶段超出预计一倍	砍范围，例如先删掉 5.7 节，而不是加班

卡住或情绪很低的时候

停一天，什么都不做。这份计划每一步都很小，没有哪一步是今天必须完成的。回来时只看下一个没打勾的格子，做完它就够了。